

POSTGRESQL + PATRONI ON KUBERNETES

TROUBLESHOOTING LAB MANUAL

Zalando/Postgres Operator - Hands-on Failure Simulation, Diagnosis, Recovery and Validation

Namespace	postgres-ha
PostgreSQL cluster resource	acid-postgres-ha
Patroni pods	acid-postgres-ha-0, acid-postgres-ha-1, acid-postgres-ha-2
Primary service	acid-postgres-ha : 5432
Replica service	acid-postgres-ha-repl : 5432
Connection pooler	acid-postgres-ha-pooler : 5432 (if deployed)
Patroni REST API	8008 inside Patroni/Spilo pods
Audience	PostgreSQL DBA / Kubernetes Administrator / SRE / Corporate Training

Environment Assumption: This manual is written for PostgreSQL managed by Patroni inside Kubernetes, not for Patroni installed as systemd services on Ubuntu VMs. Kubernetes/Operator controls pod lifecycle. Patroni usually uses Kubernetes objects/API as its distributed configuration store (DCS).

1. Safety Rules and Lab Conventions

- Run destructive fault-injection only in a disposable lab namespace. Do not run delete/scale/reinit/PVC operations in production without an approved recovery plan.
- Before every scenario, identify the current Patroni leader and replicas. Never assume pod -0 is the primary.
- Do not manually start PostgreSQL with systemctl inside containers. Patroni manages PostgreSQL and Kubernetes manages the pod.
- Do not delete a PVC to repair a replica unless the lab explicitly instructs it and the target is confirmed to be a replica.
- Use Kubernetes labels/endpoints and Patroni REST state together. A Running pod is not proof that PostgreSQL is primary/replica healthy.
- When troubleshooting authentication, do not paste real production passwords into command history. Use decoded lab secrets or environment variables.

Naming: Commands below use namespace postgres-ha and pod names acid-postgres-ha-0/1/2. If a pod has been recreated, run `kubectl get pods -n postgres-ha -o wide` and use the current names.

2. Baseline Architecture and Golden Health Check

2.1 Discover all Kubernetes objects

```
kubectl get postgresql -n postgres-ha
kubectl get pods -n postgres-ha -o wide
kubectl get svc -n postgres-ha
kubectl get endpoints -n postgres-ha
kubectl get pvc -n postgres-ha
kubectl get secrets -n postgres-ha
kubectl get events -n postgres-ha --sort-by=.lastTimestamp | tail -40
```

2.2 Identify leader and replicas from Patroni

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

If patronictl needs an explicit config path in the image:

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl -c /home/postgres/postgres.yml list
```

Do not assume: The current leader can be acid-postgres-ha-0, -1, or -2. Patroni can move leadership after failover or switchover.

2.3 Verify role from PostgreSQL itself

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- psql -U postgres -d postgres -c 'select inet_server_addr(), pg_is_in_recovery(), now();'
```

```
kubectl exec -n postgres-ha acid-postgres-ha-1 -- psql -U postgres -d postgres -c 'select inet_server_addr(), pg_is_in_recovery(), now();'
```

```
kubectl exec -n postgres-ha acid-postgres-ha-2 -- psql -U postgres -d postgres -c 'select inet_server_addr(), pg_is_in_recovery(), now();'
```

Expected Result: Exactly one pod should return `pg_is_in_recovery() = false`. Replicas should return true.

2.4 Check Patroni REST API from each pod

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- curl -s http://127.0.0.1:8008/
```

```
kubectl exec -n postgres-ha acid-postgres-ha-1 -- curl -s http://127.0.0.1:8008/
```

```
kubectl exec -n postgres-ha acid-postgres-ha-2 -- curl -s http://127.0.0.1:8008/
```

2.5 Check primary/replica Services and Endpoints

```
kubectl get svc acid-postgres-ha acid-postgres-ha-repl -n postgres-ha -o wide
kubectl get endpoints acid-postgres-ha acid-postgres-ha-repl -n postgres-ha -o yaml
```

```
# Modern clusters may also expose EndpointSlices:
kubectl get endpointslice -n postgres-ha | grep acid-postgres-ha
```

Expected Result: The primary service should resolve to the current primary endpoint; the replica service should resolve to replica endpoints according to operator labels.

3. Pod is Pending

Objective: Diagnose a Patroni pod that Kubernetes cannot schedule or mount.

Symptoms

- Pod remains Pending.
- Patroni/PostgreSQL never starts because the container has not been scheduled.

Diagnosis - run in this order

Check pod and events

```
kubectl get pods -n postgres-ha -o wide
kubectl describe pod -n postgres-ha acid-postgres-ha-2
```

Check node capacity and taints

```
kubectl get nodes -o wide
kubectl describe node <NODE_NAME> | egrep -i 'Taints|Allocatable|Allocated resources' -A20
```

Check PVC binding

```
kubectl get pvc -n postgres-ha
kubectl describe pvc -n postgres-ha <PVC_NAME>
```

Recovery

Fix the actual scheduler/storage cause

```
# Examples: free CPU/memory, correct storage class, correct node affinity/toleration.
# Do NOT repeatedly delete the pod until the event reason is understood.
```

After fixing, watch scheduling

```
kubectl get pods -n postgres-ha -w
```

Validation

```
kubectl get pods -n postgres-ha -o wide
kubectl get events -n postgres-ha --sort-by=.lastTimestamp | tail -20
```

Expected Result: All database pods reach Running/Ready and their PVCs are Bound.

4. Pod CrashLoopBackOff

Objective: Find why the Spilo/Patroni container repeatedly exits.

Symptoms

- Pod alternates between Running and CrashLoopBackOff.
- Restart count keeps increasing.

Diagnosis - run in this order

Check current status

```
kubectrl get pods -n postgres-ha
kubectrl describe pod -n postgres-ha acid-postgres-ha-1
```

Read current and previous container logs

```
kubectrl logs -n postgres-ha acid-postgres-ha-1 --tail=200
kubectrl logs -n postgres-ha acid-postgres-ha-1 --previous --tail=200
```

Inspect termination reason

```
kubectrl get pod acid-postgres-ha-1 -n postgres-ha -o jsonpath='{.status.containerStatuses[*].lastState.terminated.reason}'{"\n"}
{.status.containerStatuses[*].lastState.terminated.exitCode}'{"\n"}
```

Recovery

Correct the error indicated by logs

Typical causes: invalid Patroni bootstrap config, bad secret, broken/missing volume, permission error, OOMKilled.

Let the operator/Kubernetes recreate or restart the pod

```
kubectrl delete pod -n postgres-ha acid-postgres-ha-1
```

Validation

```
kubectrl get pod -n postgres-ha acid-postgres-ha-1 -w
```

Expected Result: Pod stabilizes at Running/Ready and restart count stops increasing.

Caution: Deleting the pod is not the repair by itself. Capture --previous logs first.

5. Replica pod deleted - automatic recreation

Objective: Observe StatefulSet/operator recovery when a replica pod disappears.

Symptoms

- Replica pod is deleted.
- Kubernetes should recreate the same ordinal and Patroni should rejoin it.

Diagnosis - run in this order

Identify a replica first

```
kubectrl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

LAB ONLY - FAULT INJECTION: Delete only a confirmed replica pod.

```
kubectl delete pod -n postgres-ha <REPLICA_POD>
```

Recovery

Watch pod recreation

```
kubectl get pods -n postgres-ha -w
```

Watch its logs during rejoin

```
kubectl logs -n postgres-ha -f <REPLICA_POD>
```

Validation

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Expected Result: Recreated member returns as Replica and pg_is_in_recovery() returns t.

6. Primary pod deleted - automatic failover

Objective: Validate Patroni/Kubernetes failover when the current primary disappears.

Symptoms

- Current primary pod is terminated.
- A healthy replica should acquire leadership, subject to Patroni policy and DCS access.

Diagnosis - run in this order

Find current leader

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

Capture primary service endpoint

```
kubectl get endpoints acid-postgres-ha -n postgres-ha -o wide
```

LAB ONLY - FAULT INJECTION: Delete ONLY the current leader pod in the lab.

```
kubectl delete pod -n postgres-ha <CURRENT_LEADER_POD>
```

Recovery

Watch pods and Patroni election

```
kubectl get pods -n postgres-ha -w
```

In another terminal:

```
kubectl exec -n postgres-ha acid-postgres-ha-1 -- patronictl list
```

Watch primary Service endpoint change

```
kubectl get endpoints acid-postgres-ha -n postgres-ha -w
```

Validation

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

```
kubectl exec -n postgres-ha <NEW_LEADER_POD> -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Expected Result: Exactly one new leader exists, primary service points to it, and `pg_is_in_recovery() = f` on the new leader.

Caution: Application RTO includes Kubernetes/Patroni election plus endpoint and connection-pool reconnection time.

7. Patroni reports unknown LSN / replication details missing

Objective: Diagnose why `patronictl` does not show expected receive/replay LSN or lag information.

Symptoms

- `patronictl` output shows unknown/missing LSN or lag.
- SQL connections may still work.

Diagnosis - run in this order

Check replica role

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -x -c "select pg_is_in_recovery(); select * from pg_stat_wal_receiver;"
```

Check primary replication view

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -x -c "select application_name,client_addr,state,sync_state,write_lsn,flush_lsn,replay_lsn from pg_stat_replication;"
```

Check Patroni REST directly

```
kubectl exec -n postgres-ha <REPLICA_POD> -- curl -s http://127.0.0.1:8008/
```

Check Patroni logs

```
kubectl logs -n postgres-ha <REPLICA_POD> --tail=250 | egrep -i 'replic|wal|lsn|error|exception'
```

Recovery

If SQL replication is healthy, treat `patronictl` display as a visibility/API issue first

Validate `pg_stat_wal_receiver` and `pg_stat_replication` before rebuilding anything.

If WAL receiver is absent, continue with the replication-disconnected scenario below

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -x -c 'select * from pg_stat_wal_receiver;'
```

Validation

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select application_name,state,sync_state,replay_lsn from pg_stat_replication;"
```

Expected Result: Healthy replicas appear in `pg_stat_replication` with `state=streaming`; use SQL views as the authoritative replication check.

8. Replica not streaming

Objective: Repair a replica whose WAL receiver is not connected to the primary.

Symptoms

- `pg_stat_wal_receiver` returns no row or status is not streaming.
- Replica lag increases.

Diagnosis - run in this order

Check WAL receiver

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -x -c 'select * from pg_stat_wal_receiver;'
```

Check primary sender

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -x -c 'select * from pg_stat_replication;'
```

Inspect replica logs

```
kubectl logs -n postgres-ha <REPLICA_POD> --tail=300 | egrep -i 'walreceiver|replication|password|hba|timeout|could not connect|requested wal'
```

Verify DNS/service connectivity from replica

```
kubectl exec -n postgres-ha <REPLICA_POD> -- getent hosts acid-postgres-ha
kubectl exec -n postgres-ha <REPLICA_POD> -- sh -c 'nc -vz acid-postgres-ha 5432 || true'
```

Recovery

Correct network/auth/WAL cause shown in logs

Do not reinitialize until you know whether the problem is transient connectivity, `pg_hba`, credential, or missing WAL.

If replica is unrecoverable, reinitialize only that member

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl reinit <PATRONI_CLUSTER_NAME> <MEMBER_NAME> --force
```

Validation

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -Atc "select status from pg_stat_wal_receiver;"
```

Expected Result: WAL receiver reports streaming and lag begins to decrease.

Caution: Reinit replaces the target replica data. Confirm member name and role before running it.

9. Replica lag is high

Objective: Measure byte/time lag and identify whether WAL generation, network, disk, or replay is the bottleneck.

Symptoms

- Replica is streaming but significantly behind.
- Read-only queries can return stale data.

Diagnosis - run in this order

Measure primary-to-replica LSN gap

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select
application_name,client_addr,state,pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),replay_lsn)) as byte_lag from
pg_stat_replication;"
```

Measure replay delay on replica

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -c "select now() pg_last_xact_replay_timestamp() as replay_delay, pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn();"
```

Check pod CPU/memory

```
kubectl top pod -n postgres-ha <REPLICA_POD>
```

Check PVC/node warnings

```
kubectl describe pod -n postgres-ha <REPLICA_POD>
kubectl get events -n postgres-ha --sort-by=.lastTimestamp | tail -30
```

Recovery

Remove resource bottleneck or heavy replay workload

Examples: increase pod resources, investigate storage latency, reduce long-running replica queries, fix network loss.

Recheck until lag converges

```
watch -n 2 "kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -Atc \"select application_name,pg_wal_lsn_diff(pg_current_wal_lsn(),replay_lsn) from pg_stat_replication;\""
```

Validation

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select application_name,state,sync_state,pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),replay_lsn)) lag from pg_stat_replication;"
```

Expected Result: Replica remains streaming and WAL byte lag trends toward zero.

10. pg_hba.conf rejects application connection

Objective: Trace a Kubernetes client connection rejected by PostgreSQL host-based authentication.

Symptoms

- Client receives no pg_hba.conf entry or authentication error.
- Example: host 10.x.x.x, user postgres, database postgres, no encryption.

Diagnosis - run in this order

Capture exact client source and error

```
kubectl logs -n postgres-ha <LEADER_POD> --tail=200 | egrep -i 'pg_hba|no pg_hba|authentication failed'
```

Show active hba file and rules

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c 'show hba_file;'
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c 'select line_number,type,database,user_name,address,auth_method,error from pg_hba_file_rules order by line_number;'
```

Check CR/operator manifest because generated files can be reconciled

```
kubectl get postgresql acid-postgres-ha -n postgres-ha -o yaml
```

Recovery

Update pg_hba through the operator-supported PostgreSQL manifest/configuration

```
kubectl edit postgresql acid-postgres-ha -n postgres-ha
```

```
# Add the required, least-privilege pg_hba rule in the manifest section used by your operator version.
```

Wait for reconciliation and validate generated rules

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c 'select
line_number,type,database,user_name,address,auth_method,error from pg_hba_file_rules order by line_number;'
```

Validation

```
psql 'host=<PRIMARY_SERVICE_IP_OR_DNS> port=5432 user=<USER> dbname=<DB> sslmode=require'
```

Expected Result: Connection succeeds using the intended SSL/authentication rule.

Caution: Do not permanently edit a generated pg_hba.conf inside the pod; operator/Patroni reconciliation can overwrite it.

11. SSL required / no encryption error

Objective: Correct a client that connects without TLS when the cluster requires SSL.

Symptoms

- Error indicates no pg_hba entry for host with no encryption.
- The same credentials may work with sslmode=require.

Diagnosis - run in this order

Verify server SSL

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c 'show ssl;'
```

Test with SSL explicitly

```
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require'
```

Inspect matching hba rules

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select line_number,type,database,user_name,address,auth_method
from pg_hba_file_rules where type in ('hostssl','host');"
```

Recovery

Configure application/exporter pool to use TLS

```
# libpq example: sslmode=require
# JDBC example: sslmode=require (or verify-full when CA/hostname validation is configured)
```

Validation

```
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c 'select current_user, ssl from
pg_stat_ssl join pg_stat_activity using(pid) where pid=pg_backend_pid()';
```

Expected Result: Client connects over SSL and no no-encryption pg_hba error remains.

12. Primary Service routes incorrectly

Objective: Diagnose cases where acid-postgres-ha does not point to the current Patroni leader.

Symptoms

- Connections to primary service reach a read-only replica or fail.
- Patroni leader is healthy but Service endpoints are stale/missing.

Diagnosis - run in this order

Find current Patroni leader

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

Inspect labels on all pods

```
kubectl get pods -n postgres-ha --show-labels
```

Inspect service selector and endpoint

```
kubectl get svc acid-postgres-ha -n postgres-ha -o yaml
kubectl get endpoints acid-postgres-ha -n postgres-ha -o yaml
```

Inspect operator logs

```
kubectl get pods -A | grep postgres-operator
# Then use the operator namespace/pod:
kubectl logs -n <OPERATOR_NAMESPACE> <OPERATOR_POD> --tail=300
```

Recovery

Do not hand-edit endpoints as a normal fix

Fix label/operator reconciliation problem. Confirm the PostgreSQL CR is valid and operator is running.

Force only a safe reconciliation trigger if needed

```
kubectl annotate postgresql acid-postgres-ha -n postgres-ha troubleshooting-reconcile="$(date +%s)" --overwrite
```

Validation

```
kubectl get endpoints acid-postgres-ha -n postgres-ha -o wide
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Expected Result: Primary service endpoint corresponds to the current leader and writes succeed.

13. Replica Service has no endpoints

Objective: Diagnose why acid-postgres-ha-repl cannot serve read-only clients.

Symptoms

- Replica service exists but Endpoints is empty.
- Read-only client cannot connect.

Diagnosis - run in this order

Inspect replicas and roles

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

Inspect pod labels

```
kubectl get pods -n postgres-ha --show-labels
```

Inspect replica service selector

```
kubectl get svc acid-postgres-ha-repl -n postgres-ha -o yaml
kubectl get endpoints acid-postgres-ha-repl -n postgres-ha -o yaml
```

Recovery

Restore healthy replicas first

If no healthy replicas exist, repair/reinit replicas; service endpoints should then be reconciled.

Verify operator is reconciling labels/endpoints

```
kubectl get events -n postgres-ha --sort-by=.lastTimestamp | tail -30
```

Validation

```
kubectl get endpoints acid-postgres-ha-repl -n postgres-ha -o wide
```

Expected Result: Replica service lists healthy replica endpoint(s).

14. Patroni REST API 8008 unavailable

Objective: Troubleshoot Patroni health API inside a pod and through a monitoring Service.

Symptoms

- curl localhost:8008 fails inside pod, or monitoring scrape reports connection refused.

Diagnosis - run in this order

Check inside pod

```
kubectl exec -n postgres-ha <POD> -- sh -c 'ss -lnt 2>/dev/null | grep 8008 || netstat -lnt 2>/dev/null | grep 8008 || true'
kubectl exec -n postgres-ha <POD> -- curl -v --connect-timeout 3 http://127.0.0.1:8008/
```

Check pod logs

```
kubectl logs -n postgres-ha <POD> --tail=250 | egrep -i 'patroni|restapi|8008|exception|error'
```

Check metrics/service if present

```
kubectl get svc patroni-metrics -n postgres-ha -o yaml
kubectl get endpoints patroni-metrics -n postgres-ha -o yaml
```

Recovery

Repair Patroni/pod startup issue shown in logs

If the Patroni process is unhealthy, let Kubernetes/operator restart the pod after the cause is corrected.

For service scrape issues, fix selector/port/NetworkPolicy

```
kubectl get networkpolicy -n postgres-ha -o yaml
```

Validation

```
kubectl exec -n postgres-ha <POD> -- curl -s http://127.0.0.1:8008/
```

Expected Result: Patroni REST returns JSON containing state/role/server_version information.

15. Kubernetes API / DCS connectivity problem

Objective: Diagnose Patroni loss of access to its Kubernetes-based distributed configuration store.

Symptoms

- Patroni logs show Kubernetes API timeouts/forbidden errors.
- Leader lock updates may fail; failover safety is affected.

Diagnosis - run in this order

Inspect Patroni logs for Kubernetes/DCS errors

```
kubectl logs -n postgres-ha <POD> --tail=400 | egrep -i 'kubernetes|dcs|forbidden|timeout|api|leader lock'
```

Check service account used by pod

```
kubectl get pod -n postgres-ha <POD> -o jsonpath='{.spec.serviceAccountName}'
```

Check authorization

```
kubectl auth can-i get pods --as=system:serviceaccount:postgres-ha:<SERVICE_ACCOUNT> -n postgres-ha
kubectl auth can-i get configmaps --as=system:serviceaccount:postgres-ha:<SERVICE_ACCOUNT> -n postgres-ha
kubectl auth can-i get endpoints --as=system:serviceaccount:postgres-ha:<SERVICE_ACCOUNT> -n postgres-ha
```

Check API reachability from pod

```
kubectl exec -n postgres-ha <POD> -- sh -c 'wget -qO- --timeout=3 https://kubernetes.default.svc 2>&1 | head | | true'
```

Recovery

Restore RBAC/API/network access

Correct Role/RoleBinding/ServiceAccount or NetworkPolicy/CNI/DNS issue. Do not force promotion while DCS state is ambiguous.

Recheck Patroni after API access returns

```
kubectl exec -n postgres-ha <POD> -- patronictl list
```

Validation

```
kubectl exec -n postgres-ha <POD> -- patronictl list
kubectl logs -n postgres-ha <POD> --tail=80
```

Expected Result: Patroni can read/update DCS again and leader/replica state stabilizes.

Caution: Loss of DCS is a cluster-safety issue. Avoid manual promotion until quorum/lock state is understood.

16. DNS failure inside namespace

Objective: Diagnose service-name resolution failures affecting database clients or replication.

Symptoms

- getent/nslookup for acid-postgres-ha fails.
- Connections by ClusterIP may work while service DNS fails.

Diagnosis - run in this order

Check service exists

```
kubectl get svc acid-postgres-ha -n postgres-ha
```

Test DNS from a database pod

```
kubectl exec -n postgres-ha <POD> -- getent hosts acid-postgres-ha
kubectl exec -n postgres-ha <POD> -- getent hosts acid-postgres-ha.postgres-ha.svc.cluster.local
```

Check CoreDNS

```
kubectl get pods -n kube-system | grep -E 'coredns|kube-dns'
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100
```

Check pod resolver

```
kubectl exec -n postgres-ha <POD> -- cat /etc/resolv.conf
```

Recovery

Repair CoreDNS/CNI or pod DNS policy issue

Do not replace service DNS permanently with pod IPs; pod IPs are ephemeral.

Validation

```
kubectl exec -n postgres-ha <POD> -- getent hosts acid-postgres-ha.postgres-ha.svc.cluster.local
```

Expected Result: Cluster DNS resolves the primary service name to the service ClusterIP.

17. NetworkPolicy blocks PostgreSQL traffic

Objective: Identify a policy that permits pods to run but blocks TCP/5432 or monitoring traffic.

Symptoms

- Connection times out instead of returning PostgreSQL authentication error.
- DNS resolves correctly.

Diagnosis - run in this order

List policies

```
kubectl get networkpolicy -n postgres-ha
kubectl describe networkpolicy -n postgres-ha
```

Test TCP from a client pod

```
kubectl run -n postgres-ha nettest --rm -it --restart=Never --image=busybox:1.36 -- sh
# Inside: nc -vz acid-postgres-ha 5432
```

Test Patroni metrics if applicable

Inside client/Prometheus pod: nc -vz patroni-metrics 8008

Recovery

Update ingress/egress rules with least privilege

```
# Allow intended application namespace/pods to TCP 5432.
# Allow monitoring namespace/pods to 8008/9187 only if required.
```

Validation

```
kubectl run -n postgres-ha nettest --rm -i --restart=Never --image=busybox:1.36 -- nc -vz acid-postgres-ha 5432
```

Expected Result: TCP connection to the intended Service/port succeeds.

18. PVC or storage mount failure

Objective: Diagnose pod startup failure caused by unbound/mounting volume.

Symptoms

- Pod is Pending/ContainerCreating.
- Events show FailedMount, Multi-Attach, or volume provisioning errors.

Diagnosis - run in this order

Inspect pod events

```
kubectl describe pod -n postgres-ha <POD>
```

Inspect PVC/PV

```
kubectl get pvc -n postgres-ha -o wide
kubectl get pv
kubectl describe pvc -n postgres-ha <PVC>
```

Inspect storage class

```
kubectl get storageclass
kubectl describe storageclass <STORAGE_CLASS>
```

Recovery

Fix CSI/storage attachment issue

```
# Examples: wait for old node detach, repair CSI driver, correct zone/topology, restore storage quota.
```

Do not create a blank replacement volume for an existing member casually

```
# Patroni reinit should be used only after confirming recovery strategy.
```

Validation

```
kubectl get pod -n postgres-ha <POD> -o wide
kubectl get pvc -n postgres-ha <PVC> -o wide
```

Expected Result: PVC is Bound, mounted, and the pod reaches Running/Ready.

19. Disk/PVC full

Objective: Recover from PostgreSQL write failures due to filesystem exhaustion.

Symptoms

- Logs contain No space left on device.
- WAL/checkpoint/temp writes fail; pod may become unhealthy.

Diagnosis - run in this order

Check filesystem inside pod

```
kubectl exec -n postgres-ha <POD> -- df -h
kubectl exec -n postgres-ha <POD> -- df -i
```

Find large PostgreSQL directories

```
kubectl exec -n postgres-ha <POD> -- sh -c 'du -xhd1 $PGDATA 2>/dev/null | sort -h | tail -20'
```

Check WAL usage

```
kubectl exec -n postgres-ha <POD> -- psql -U postgres -c "select pg_size_pretty(sum(size)) from pg_ls_waldir();"
```

Check PVC capacity

```
kubectl get pvc -n postgres-ha -o wide
```

Recovery

Expand PVC if the StorageClass supports it

```
kubectl patch pvc -n postgres-ha <PVC> -p '{"spec":{"resources":{"requests":{"storage":"<NEW_SIZE>"}}}}'
```

Remove only known disposable files / fix retention cause

```
# Never rm files from PGDATA/pg_wal manually to make space.
```

Confirm filesystem resize

```
kubectl exec -n postgres-ha <POD> -- df -h
```

Validation

```
kubectl exec -n postgres-ha <POD> -- psql -U postgres -c 'select now();'
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

Expected Result: PostgreSQL can write again and the member returns healthy.

Caution: Never manually delete WAL files from pg_wal.

20. OOMKilled / memory pressure

Objective: Diagnose a PostgreSQL pod killed by Kubernetes memory limits or node pressure.

Symptoms

- Last termination reason is OOMKilled.
- Pod restarts under heavy queries/load.

Diagnosis - run in this order

Check termination reason

```
kubectl describe pod -n postgres-ha <POD> | egrep -i 'OOMKilled|Reason|Limits|Requests' -A5
```

Check current usage

```
kubectl top pod -n postgres-ha <POD> --containers
```

Inspect PostgreSQL memory settings

```
kubectl exec -n postgres-ha <POD> -- psql -U postgres -c "show shared_buffers; show work_mem; show maintenance_work_mem; show max_connections;"
```

Check node pressure

```
kubectl describe node <NODE> | egrep -i 'MemoryPressure|Allocated resources' -A20
```

Recovery

Right-size pod memory requests/limits through the PostgreSQL custom resource

```
kubectl edit postgresql acid-postgres-ha -n postgres-ha
```

Tune database concurrency/memory if needed

work_mem is per operation, not a fixed cluster total. Review max_connections and pooler use.

Validation

```
kubectl get pod -n postgres-ha <POD>
kubectl top pod -n postgres-ha <POD> --containers
```

Expected Result: Pod stays Running without new OOMKilled events under expected load.

21. CPU throttling / performance slowdown

Objective: Detect CPU limits or saturation that make Patroni/PostgreSQL slow without crashing.

Symptoms

- Queries and replication slow down.
- Pod remains Running but CPU is at/near limit.

Diagnosis - run in this order

Check pod CPU

```
kubectl top pod -n postgres-ha
```

Show requests/limits

```
kubectl get pod -n postgres-ha <POD> -o jsonpath='{.spec.containers[*].resources}{"\n"}'
```

Check active workload

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select pid,username,state,wait_event_type,wait_event,left(query,100) from pg_stat_activity where state<>'idle' order by query_start;"
```

Recovery

Right-size CPU via operator manifest

```
kubectl edit postgresql acid-postgres-ha -n postgres-ha
```

Investigate expensive SQL separately

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select * from pg_stat_activity where state <> 'idle';"
```

Validation

```
kubectl top pod -n postgres-ha
```

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

Expected Result: CPU has headroom and replica lag/query latency return toward baseline.

22. Missing WAL / requested WAL segment already removed

Objective: Recover a replica that cannot catch up because required WAL is no longer available.

Symptoms

- Replica log says requested WAL segment has already been removed.
- Streaming cannot resume normally.

Diagnosis - run in this order

Confirm exact WAL error

```
kubectl logs -n postgres-ha <REPLICA_POD> --tail=400 | egrep -i 'requested wal|removed|wal segment|could not receive'
```

Check slots on primary

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select slot_name,slot_type,active,restart_lsn,wal_status from pg_replication_slots;"
```

Check WAL retention

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "show wal_keep_size; show max_slot_wal_keep_size;"
```

Recovery

Reinitialize only the broken replica

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl reinit <PATRONI_CLUSTER_NAME> <MEMBER_NAME> --force
```

Review WAL retention and replication slot policy after recovery

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select * from pg_replication_slots;"
```

Validation

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
```

Expected Result: Replica is rebuilt from a current base copy and returns to streaming.

23. Former primary cannot rejoin after failover

Objective: Handle timeline divergence after an old primary returns.

Symptoms

- Former leader starts but cannot follow current timeline.
- Logs mention timeline divergence or pg_rewind.

Diagnosis - run in this order

Inspect former-primary logs

```
kubectl logs -n postgres-ha <FORMER_LEADER_POD> --tail=400 | egrep -i 'timeline|rewind|diverg|history|reinitialize'
```

Check current cluster roles

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
```

Check rewind prerequisites

```
kubectl exec -n postgres-ha <CURRENT_LEADER_POD> -- psql -U postgres -c "show wal_log_hints;"
```

Recovery

Allow Patroni to use pg_rewind when configured and possible

Patroni/Spilo normally handles former-primary rewind automatically when prerequisites are met.

If it cannot be repaired, reinitialize that former leader as a replica

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl reinit <PATRONI_CLUSTER_NAME> <FORMER_LEADER_MEMBER> --force
```

Validation

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
```

Expected Result: Former primary returns as Replica on the current timeline.

24. Manual switchover for maintenance

Objective: Move leadership cleanly to another healthy replica before planned maintenance.

Symptoms

- You need to drain/restart the current primary without waiting for failure.

Diagnosis - run in this order

Check all members and lag

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
```

Confirm candidate is replica and healthy

```
kubectl exec -n postgres-ha <CANDIDATE_POD> -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Recovery

Run Patroni switchover interactively

```
kubectl exec -it -n postgres-ha <HEALTHY_POD> -- patronictl switchover <PATRONI_CLUSTER_NAME>
```

Follow prompts for current leader and candidate

```
# Select a healthy, sufficiently caught-up replica.
```

Validation

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
kubectl get endpoints acid-postgres-ha -n postgres-ha -o wide
```

Expected Result: Selected candidate becomes Leader and primary Service endpoint follows it.

25. Manual failover when automatic election did not complete

Objective: Use controlled Patroni failover only after understanding why the leader is unavailable.

Symptoms

- Leader is unavailable and no automatic promotion occurred.

Diagnosis - run in this order

Check Patroni state on surviving pods

```
kubectl exec -n postgres-ha <SURVIVING_POD> -- patronictl list
```

Check Kubernetes/DCS/API errors

```
kubectl logs -n postgres-ha <SURVIVING_POD> --tail=300 | egrep -i 'dcs|kubernetes|leader|lock|failover|lag'
```

Check candidate lag

```
kubectl exec -n postgres-ha <CANDIDATE_POD> -- psql -U postgres -c "select now()-pg_last_xact_replay_timestamp(),
pg_last_wal_replay_lsn();"

```

Recovery

Fence/confirm the old primary cannot accept writes if state is ambiguous

```
# Required to avoid two writable primaries outside Patroni's protection.
```

Run failover

```
kubectl exec -it -n postgres-ha <SURVIVING_POD> -- patronictl failover <PATRONI_CLUSTER_NAME>
```

Validation

```
kubectl exec -n postgres-ha <SURVIVING_POD> -- patronictl list
kubectl get endpoints acid-postgres-ha -n postgres-ha -o wide
```

Expected Result: Exactly one leader exists and the primary service points to it.

Caution: Do not use forced failover as a shortcut for a Kubernetes API/DCS outage or unknown network partition.

26. Reinitialize a broken replica

Objective: Rebuild a replica under Patroni control without deleting the wrong PVC/member.

Symptoms

- Replica repeatedly fails streaming or has unrecoverable timeline/WAL problems.

Diagnosis - run in this order

Confirm member roles

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
```

Save logs before destructive recovery

```
kubectl logs -n postgres-ha <REPLICA_POD> --tail=1000 > replica-before-reinit.log
```

Confirm target is not leader

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Recovery

Reinitialize with Patroni

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl reinit <PATRONI_CLUSTER_NAME> <REPLICA_MEMBER> --force
```

Monitor clone/rejoin

```
kubectl logs -n postgres-ha -f <REPLICA_POD>
```

Validation

```
kubectl exec -n postgres-ha <HEALTHY_POD> -- patronictl list
```

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Expected Result: Target member is rebuilt and returns as Replica with `pg_is_in_recovery() = t`.

Caution: Reinit is destructive to the selected replica data. Verify member name twice.

27. Secret/password mismatch

Objective: Troubleshoot PostgreSQL credentials stored in Kubernetes Secrets.

Symptoms

- psql reports password authentication failed.
- A stale manually copied password may differ from the operator-managed secret.

Diagnosis - run in this order

List relevant secrets

```
kubectl get secrets -n postgres-ha | grep -E 'postgres|bank|acid'
```

Inspect secret keys without printing values

```
kubectl get secret -n postgres-ha <SECRET_NAME> -o json | python3 -c "import sys,json; print('\n'.join(json.load(sys.stdin).get('data', {})).keys())"
```

Use password from secret in a temporary shell variable

```
export PGPASSWORD="$(kubectl get secret -n postgres-ha <SECRET_NAME> -o jsonpath='{.data.password}' | base64 -d)"
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c 'select current_user;'
unset PGPASSWORD
```

Recovery**Update the consuming application Secret/reference rather than hard-coding password**

Keep operator-managed user credentials synchronized with the application deployment.

Validation

```
export PGPASSWORD="$(kubectl get secret -n postgres-ha <SECRET_NAME> -o jsonpath='{.data.password}' | base64 -d)"
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c 'select current_user;'
unset PGPASSWORD
```

Expected Result: Authentication succeeds using the current Kubernetes Secret.

28. Connection pooler/PgBouncer cannot connect

Objective: Troubleshoot acid-postgres-ha-pooler when direct PostgreSQL works.

Symptoms

- Direct connection to acid-postgres-ha succeeds but pooler connection fails.
- Pooler pod may be unhealthy or backend credentials/routing wrong.

Diagnosis - run in this order**Check pooler objects**

```
kubectl get pods,svc -n postgres-ha | grep pooler
```

Check pooler logs

```
kubectl logs -n postgres-ha -l application=spilo,connection-pooler=true --tail=200 2>/dev/null || kubectl logs -n postgres-ha
<POOLER_POD> --tail=200
```

Test primary directly

```
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require'
```

Test pooler

```
psql 'host=acid-postgres-ha-pooler.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require'
```

Recovery**Restart only after capturing logs**

```
kubectl delete pod -n postgres-ha <POOLER_POD>
```

Verify pooler Secret/config references

```
kubectl describe pod -n postgres-ha <POOLER_POD>
```

Validation

```
kubectl get endpoints acid-postgres-ha-pooler -n postgres-ha -o wide
psql 'host=acid-postgres-ha-pooler.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c 'select 1;'
```

Expected Result: Pooler Service has endpoints and client query succeeds.

29. postgres-exporter shows pg_up = 0

Objective: Troubleshoot monitoring when PostgreSQL itself is healthy but exporter reports down.

Symptoms

- Prometheus metric pg_up is 0.
- Exporter logs show connection/auth/SSL errors.

Diagnosis - run in this order

Check exporter pod/service

```
kubectl get pods,svc -n postgres-ha | grep exporter
```

Read exporter logs

```
kubectl logs -n postgres-ha <EXPORTER_POD> --tail=250
```

Inspect datasource environment without exposing secrets

```
kubectl describe pod -n postgres-ha <EXPORTER_POD> | egrep -i 'DATA_SOURCE|PGHOST|PGPORT|PGUSER|Secret' -A2
```

Test exporter credentials manually from a postgres pod or debug pod

```
export PGPASSWORD="$(kubectl get secret -n postgres-ha <SECRET> -o jsonpath='{.data.password}' | base64 -d)"
psql 'host=acid-postgres-ha port=5432 user=<EXPORTER_USER> dbname=postgres sslmode=require' -c 'select 1;'
unset PGPASSWORD
```

Recovery

Correct DATA_SOURCE_NAME/Secret and SSL mode

For clusters requiring TLS, include sslmode=require (or verify-full when CA is configured).

Restart exporter deployment after config change

```
kubectl rollout restart deployment -n postgres-ha <EXPORTER_DEPLOYMENT>
```

Validation

```
kubectl port-forward -n postgres-ha svc/postgres-exporter 9187:80
# In another terminal:
curl -s http://127.0.0.1:9187/metrics | grep '^pg_up'
```

Expected Result: pg_up 1 is exposed and Prometheus can scrape the exporter.

30. Prometheus not scraping PostgreSQL/Patroni

Objective: Differentiate exporter health from Prometheus discovery/scrape configuration.

Symptoms

- Exporter /metrics works locally but target is absent/down in Prometheus.

Diagnosis - run in this order

Verify Service/endpoints

```
kubectl get svc,endpoints -n postgres-ha postgres-exporter patroni-metrics 2>/dev/null
```

Check labels used by ServiceMonitor/PodMonitor if Prometheus Operator is used

```
kubectl get servicemonitor,podmonitor -A 2>/dev/null | grep -E 'postgres|patroni'
```

Check Prometheus targets/config mechanism

```
# Port-forward Prometheus and inspect Status -> Targets, or inspect its scrape configuration/ServiceMonitor.
```

Test network path from Prometheus namespace

```
kubectl get networkpolicy -A | grep -E 'prometheus|postgres-ha'
```

Recovery

Add/fix ServiceMonitor or scrape_config to match service labels and ports

```
# Ensure the port name/path matches /metrics and the namespaceSelector includes postgres-ha.
```

Validation

```
kubectl get endpoints -n postgres-ha postgres-exporter patroni-metrics 2>/dev/null
```

Expected Result: Monitoring services have endpoints and Prometheus target status becomes UP.

31. Operator is not reconciling PostgreSQL CR

Objective: Troubleshoot desired-state changes that are not reflected in pods/services.

Symptoms

- kubectl edit postgresql succeeds but no rollout/change occurs.
- Service labels/replica count stay stale.

Diagnosis - run in this order

Inspect CR status

```
kubectl get postgresql acid-postgres-ha -n postgres-ha -o yaml
```

Find operator pod

```
kubectl get pods -A | grep postgres-operator
```

Read operator logs

```
kubectl logs -n <OPERATOR_NAMESPACE> <OPERATOR_POD> --tail=400 | egrep -i 'acid-postgres-ha|error|warning|reconcile'
```

Check CRD and events

```
kubectl get crd | grep postgresql
```

```
kubectl get events -n postgres-ha --sort-by=.lastTimestamp | tail -40
```

Recovery

Fix invalid CR/RBAC/operator issue shown in logs

Do not hand-edit StatefulSet as a permanent workaround; operator may revert it.

Verify reconciliation

```
kubectrl get pods -n postgres-ha -w
```

Validation

```
kubectrl get postgresql acid-postgres-ha -n postgres-ha
kubectrl get pods -n postgres-ha
```

Expected Result: Operator observes the CR and converges Kubernetes objects to desired state.

32. Node failure / pod eviction

Objective: Validate cluster behavior when the Kubernetes node hosting a Patroni member fails.

Symptoms

- Pod becomes Unknown/Terminating/Evicted or node is NotReady.
- If leader is affected, Patroni should promote a healthy replica that can access DCS/storage.

Diagnosis - run in this order

Map pods to nodes

```
kubectrl get pods -n postgres-ha -o wide
```

Check node conditions

```
kubectrl get nodes
kubectrl describe node <FAILED_NODE> | egrep -i 'Ready|Pressure|Taint' -A5
```

Check Patroni roles from surviving pod

```
kubectrl exec -n postgres-ha <SURVIVING_POD> -- patronictl list
```

Check volume topology/attachment

```
kubectrl get pvc -n postgres-ha -o wide
kubectrl get events -n postgres-ha --sort-by=.lastTimestamp | tail -50
```

Recovery

Restore node or let workload reschedule according to StatefulSet/storage topology

If volume is zonal/single-attach, attachment/detachment can extend recovery time.

If the failed node hosted leader, verify Service endpoint follows new leader

```
kubectrl get endpoints acid-postgres-ha -n postgres-ha -w
```

Validation

```
kubectrl exec -n postgres-ha <SURVIVING_POD> -- patronictl list
kubectrl get pods -n postgres-ha -o wide
```

Expected Result: One leader remains available and failed member eventually rejoins/recreates when infrastructure permits.

33. Read-only transaction on primary service

Objective: Determine whether a client accidentally reached a replica or is inside a read-only transaction.

Symptoms

- INSERT/UPDATE fails with cannot execute in a read-only transaction.

Diagnosis - run in this order

Check server role from the same client connection

```
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c "select pg_is_in_recovery(),
current_setting('transaction_read_only'), inet_server_addr();"

```

Check primary service endpoint

```
kubectl get endpoints acid-postgres-ha -n postgres-ha -o wide

```

Check Patroni roles

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list

```

Check transaction/default setting

```
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c "show
default_transaction_read_only; show transaction_read_only;"

```

Recovery

If connected to replica, repair service/connection target

```
# Application writes must use acid-postgres-ha, not acid-postgres-ha-repl.

```

If transaction itself is read-only, start a normal transaction

```
ROLLBACK;
BEGIN;
-- retry intended write

```

Validation

```
psql 'host=acid-postgres-ha.postgres-ha.svc port=5432 user=<USER> dbname=<DB> sslmode=require' -c "select pg_is_in_recovery(),
current_setting('transaction_read_only');"

```

Expected Result: Write connection reports pg_is_in_recovery()=f and transaction_read_only=off.

34. Too many connections

Objective: Recover when PostgreSQL reaches max_connections and clients are rejected.

Symptoms

- FATAL: remaining connection slots are reserved or too many clients.

- Pooler may also be saturated.

Diagnosis - run in this order

Check connection counts

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select count(*) total, state, username from pg_stat_activity group by state,username order by total desc;"
```

Check max_connections

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c 'show max_connections;'
```

Find long idle sessions

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select pid,username,application_name,client_addr,state,now()-state_change idle_for from pg_stat_activity where state like 'idle%' order by state_change;"
```

Check pooler

```
kubectl get pods,svc -n postgres-ha | grep pooler
```

Recovery

Terminate only confirmed disposable/stale sessions if necessary

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select pg_terminate_backend(pid) from pg_stat_activity where state='idle' and pid<>pg_backend_pid() and now()-state_change > interval '30 minutes';"
```

Use/size pooler and application pools appropriately

Avoid simply raising max_connections without memory sizing.

Validation

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -c "select count(*) from pg_stat_activity;"
```

Expected Result: New connections succeed and active connection count is below safe capacity.

35. End-to-End Troubleshooting Decision Matrix

Symptom	First checks	Likely area
Pod Pending	kubectl describe pod; PVC/events	Scheduler/resources/storage
CrashLoopBackOff	kubectl logs --previous	Container/Patroni/bootstrap error
Cannot write	pg_is_in_recovery; primary Service endpoint	Wrong service / no leader / read-only txn
Replica lag	pg_stat_replication; pg_stat_wal_receiver	Network/disk/replay/resource bottleneck
No pg_hba entry	pg_hba_file_rules; client source; sslmode	Auth rule / SSL mismatch
Connection timeout	DNS; endpoints; NetworkPolicy	Service/CNI/policy
pg_up 0	exporter logs + manual psql with same secret	Exporter DSN/auth/SSL
No leader after failure	Patroni logs + Kubernetes DCS/API access	DCS/API/RBAC/lag policy
Requested WAL removed	replica logs + slots/WAL retention	Reinit replica; fix retention
Pod OOMKilled	describe pod; kubectl top; PG memory settings	Resource limits / workload

PVC full	df -h; PVC capacity; WAL usage	Expand storage / fix retention
Primary service stale	pod labels + svc selector + endpoints + operator logs	Operator reconciliation/labels

36. Rapid Health-Check Command Sheet

Objects

```
kubectl get postgresql,pods,svc,endpoints,pvc -n postgres-ha -o wide
```

Events

```
kubectl get events -n postgres-ha --sort-by=.lastTimestamp | tail -40
```

Patroni roles

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- patronictl list
```

PostgreSQL roles

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

```
kubectl exec -n postgres-ha acid-postgres-ha-1 -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

```
kubectl exec -n postgres-ha acid-postgres-ha-2 -- psql -U postgres -Atc 'select pg_is_in_recovery();'
```

Primary replication view

```
kubectl exec -n postgres-ha <LEADER_POD> -- psql -U postgres -x -c 'select
application_name,client_addr,state,sync_state,write_lsn,flush_lsn,replay_lsn from pg_stat_replication;'
```

Replica WAL receiver

```
kubectl exec -n postgres-ha <REPLICA_POD> -- psql -U postgres -x -c 'select * from pg_stat_wal_receiver;'
```

Services/endpoints

```
kubectl get endpoints acid-postgres-ha acid-postgres-ha-repl -n postgres-ha -o wide
```

Patroni REST

```
kubectl exec -n postgres-ha <POD> -- curl -s http://127.0.0.1:8008/
```

Logs

```
kubectl logs -n postgres-ha <POD> --tail=250
```

```
kubectl logs -n postgres-ha <POD> --previous --tail=250
```

Resource usage

```
kubectl top pod -n postgres-ha
```

Storage

```
kubectl get pvc -n postgres-ha -o wide
```

```
kubectl exec -n postgres-ha <POD> -- df -h
```

37. Lab Completion Checklist

- I can identify the Patroni leader without assuming pod ordinal.
- I can distinguish Kubernetes pod health from PostgreSQL/Patroni role health.
- I can verify primary and replica Service endpoints.
- I can diagnose replication using `pg_stat_replication` and `pg_stat_wal_receiver`.
- I can troubleshoot `pg_hba`/SSL/authentication without permanently editing generated files inside a pod.
- I can diagnose DCS/Kubernetes API/RBAC problems safely.
- I can reinitialize only a confirmed broken replica.

- I can perform planned switchover and validate Service endpoint movement.
- I can troubleshoot PVC full, OOMKilled, Pending, CrashLoopBackOff, DNS and NetworkPolicy problems.
- I can validate pooler/exporter/Prometheus problems separately from PostgreSQL health.

Training Practice: Repeat the failure scenarios in a disposable cluster until diagnosis is based on evidence (events, logs, SQL views, Patroni state and endpoints), not on pod names or assumptions.